

Product Requirements Document (PRD)

InfoBridge: Smart Access to Public Services

April 15, 2026

1 Abstract

Accessing government service information is often challenging due to unstructured and lengthy documentation. This project presents InfoBridge, a Retrieval-Augmented Generation (RAG) based chatbot that retrieves relevant information from official government PDFs and generates accurate, source-backed responses. The system aims to improve accessibility, reduce user effort, and provide reliable answers in a conversational interface.

2 Introduction

Government services such as passport application, voter registration, and taxation involve extensive documentation. Users often struggle to navigate these documents and extract relevant information efficiently. This project addresses the problem by building an AI-powered chatbot that enables users to query government services and receive precise, context-aware answers.

3 Objectives

- Develop a chatbot for querying government service information.
- Implement a RAG-based pipeline for accurate answer generation.
- Provide source-backed responses to ensure reliability.
- Improve accessibility through a user-friendly interface.

4 Scope

Included Services:

- Passport services
- Voter ID services
- Driving Licence
- Income Tax
- Ayushman Bharat

Out of Scope:

- Real-time API integration

- User authentication
- Transaction processing

5 System Overview

The system follows a Retrieval-Augmented Generation (RAG) pipeline:

1. Collect government PDFs
2. Extract and preprocess text
3. Split text into chunks
4. Generate embeddings
5. Store embeddings in vector database
6. Retrieve relevant chunks for query
7. Generate response using LLM

6 System Architecture

PDFs → Text → Chunks → Embeddings → Vector DB
↓
User Query → Embedding → Retrieval → LLM → Answer + Sources

7 Dataset Description

The dataset consists of official government PDF documents collected from public portals. Approximately 20–40 PDFs are used across multiple services. Each document is processed into smaller text chunks (300–500 words) with metadata including source and service type.

8 Methodology

8.1 Data Preprocessing

- Extract text using PyPDF2
- Remove noise such as headers and footers
- Split text into overlapping chunks

8.2 Embedding Generation

- Model: Sentence Transformers (MiniLM)
- Convert text chunks into vector embeddings

8.3 Vector Storage

- Store embeddings using FAISS
- Enable fast similarity search

8.4 Query Processing

- Convert user query into embedding
- Retrieve top-k similar chunks

8.5 Answer Generation

- Use Gemini API for response generation
- Provide context-aware answers
- Restrict responses to retrieved content

9 Functional Requirements

- Accept user queries through chat interface
- Retrieve relevant document segments
- Generate accurate responses using LLM
- Display source references
- Support multi-service queries
- Optional: Language toggle (English/Hindi)

10 Non-Functional Requirements

- Response latency < 3 seconds
- High accuracy and reliability
- Scalable architecture
- User-friendly interface

11 Evaluation Metrics

- **Accuracy:** Correctness of responses
- **Relevance:** Quality of retrieved documents
- **Latency:** Response time
- **User Satisfaction:** Feedback-based evaluation

12 Technology Stack

- Frontend: Streamlit
- Backend: Python
- Embeddings: Sentence Transformers (MiniLM)
- Vector DB: FAISS

- LLM: Gemini API
- PDF Parsing: PyPDF2

13 Workflow

13.1 Data Preparation

Collect and preprocess PDF documents.

13.2 Embedding Pipeline

Generate embeddings and store in vector database.

13.3 Query Pipeline

Retrieve relevant chunks based on user query.

13.4 Response Generation

Generate final answer with LLM.

14 Team Responsibilities

- Data Team: Data collection and preprocessing
- Backend Team: Embeddings and retrieval
- LLM Team: Prompt engineering
- Frontend Team: UI development

15 Detailed Team Responsibilities and Execution Plan

15.1 Data Team (2 Members)

Objective: Collect and preprocess dataset from government PDFs.

Tasks:

1. Identify and download official PDFs from government portals:
 - Passport (passportindia.gov.in)
 - Voter ID (eci.gov.in)
 - Driving Licence (parivahan.gov.in)
 - Income Tax (incometax.gov.in)
 - Ayushman Bharat (nha.gov.in)

2. Organize files into structured folders:

```
data/passport/
data/voter/
data/driving/
data/tax/
data/health/
```

3. Extract text from PDFs using PyPDF2.
4. Clean extracted text:
 - Remove headers, footers, and page numbers
 - Remove unnecessary symbols and formatting issues
5. Validate extracted text quality.

Output: Clean text data ready for chunking.

15.2 Backend Team (2 Members)

Objective: Build embedding and retrieval pipeline.

Tasks:

1. Implement text chunking:
 - Chunk size: 300–500 words
 - Overlap: 50 words
2. Add metadata to each chunk:
 - Source file name
 - Service type
3. Generate embeddings using Sentence Transformers (MiniLM).
4. Store embeddings in FAISS vector database.
5. Implement retrieval function:
 - Convert query to embedding
 - Retrieve top-k relevant chunks
6. Optimize retrieval performance (tune k value).

Output: Working retrieval pipeline.

15.3 LLM Team (1–2 Members)

Objective: Generate accurate responses using LLM.

Tasks:

1. Integrate Gemini (or Llama) API.
2. Design prompt template:
 - Restrict answers to provided context
 - Handle missing information cases
3. Implement response generation:
 - Combine retrieved chunks into context
 - Generate answer using LLM

4. Add formatting:
 - Clear answer structure
 - Bullet points for readability
5. Implement optional features:
 - Language toggle (English/Hindi)
 - Error handling for empty responses

Output: Context-aware answer generation module.

15.4 Frontend Team (1–2 Members)

Objective: Build user interface and integrate system.

Tasks:

1. Develop Streamlit-based chat interface:
 - Input box for user queries
 - Display chatbot responses
2. Display retrieved sources:
 - Show document names
 - Optional: highlight relevant text
3. Implement chat history:
 - Store previous queries and responses
4. Add UI enhancements:
 - Clean layout
 - Responsive design
5. Integrate backend and LLM pipeline:
 - Connect query input to retrieval function
 - Display generated answers

Output: Fully functional web application.

15.5 Integration and Testing (All Members)

Tasks:

1. Integrate all modules:
 - UI → Retrieval → LLM → Output
2. Perform testing:
 - Test queries from all services
 - Validate accuracy and relevance

3. Debug issues:

- Fix incorrect retrieval
- Improve prompt design

4. Optimize performance:

- Reduce latency
- Improve response clarity

Output: Final working system ready for deployment.

16 Timeline

- Day 1: Data collection
- Day 2: Embeddings and retrieval
- Day 3: LLM integration
- Day 4: UI and testing

17 Challenges

- Handling noisy PDF data
- Avoiding hallucinations
- Ensuring relevant retrieval

18 Future Work

- Multi-language support
- Voice-based interaction
- Mobile deployment
- Integration with APIs

19 Conclusion

InfoBridge demonstrates an effective application of RAG systems for improving access to government services. It enables users to interact with complex documents through a simple conversational interface, ensuring accuracy and reliability.